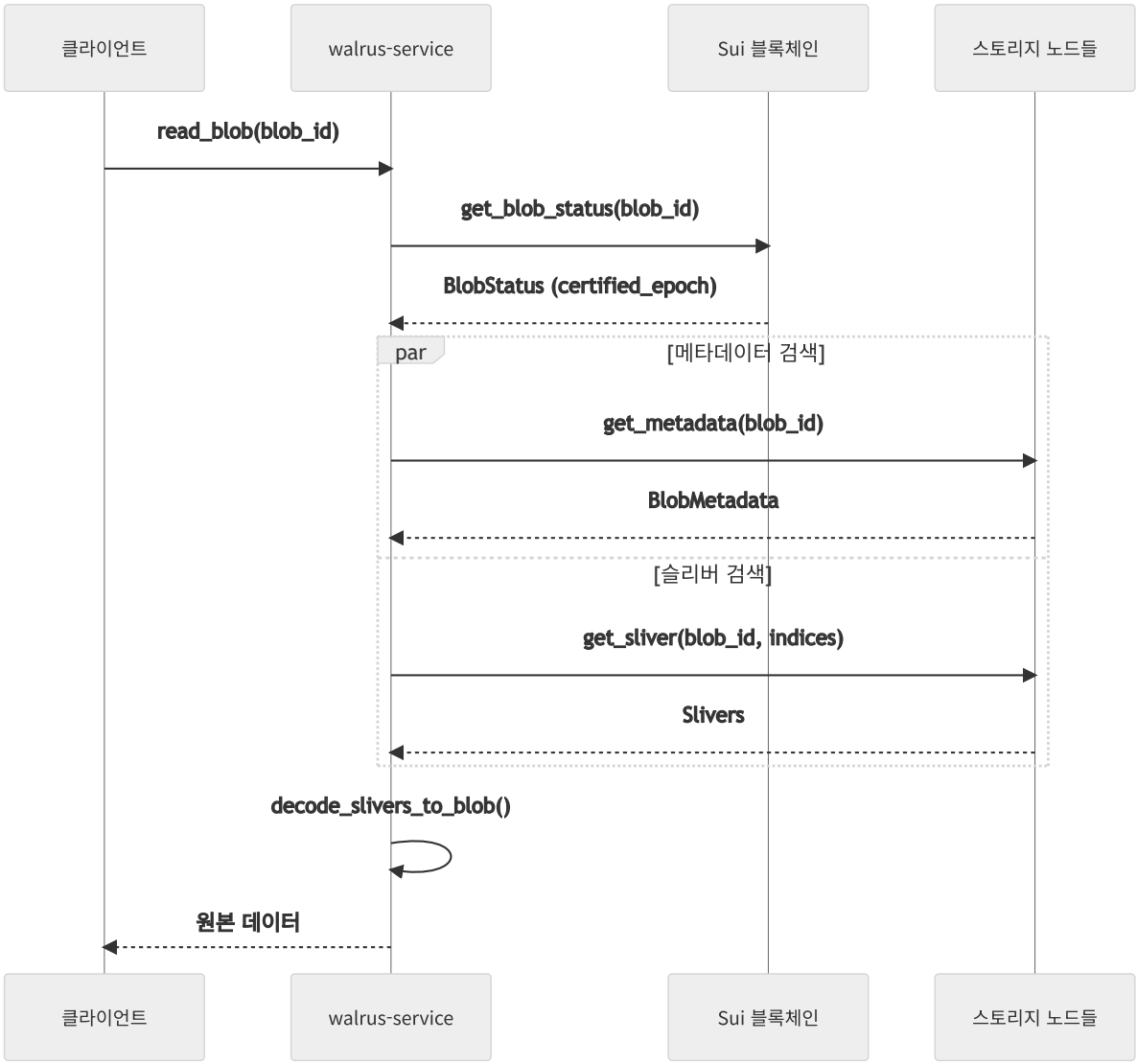
데이터 검색 및 복구 프로세스 분석

개요

Walrus에서 데이터 검색과 복구는 RedStuff 인코딩 알고리즘의 핵심 기능입니다. 이 문서는 블롭 ID를 통한 데이터 검색 과정, 슬리버 수집과 RaptorQ 디코딩을 통한 원본 데이터 복원, 노드 장애 시 슬리버 복구 메커니즘, 비잔틴 노드 탐지 및 대응 전략, 그리고 성능 최적화 기법들을 상세히 분석합니다.

1. 블롭 ID를 통한 데이터 검색 과정

1.1 검색 프로세스 개요



1.2 블롭 상태 확인

데이터 검색의 첫 번째 단계는 블롭의 상태를 확인하는 것입니다:

```
// BlobStatus 열거형 정의

pub  enum BlobStatus {
```

```
/// 블롭이 존재하지 않음
Nonexistent,
/// 블롭 ID가 무효로 표시됨
Invalid { event: EventID },
/// 영구 상태의 블롭
Permanent {
end_epoch: Epoch,
is_certified: bool,
status_event: EventID,
deletable_counts: DeletableCounts,
initial_certified_epoch: Option<Epoch>,
},
/// 삭제 가능한 상태의 블롭
Deletable {
initial_certified_epoch: Option<Epoch>,
deletable_counts: DeletableCounts,
},
}
```

1.3 인증된 에포크 결정

```
async fn get_blob_status_and_certified_epoch(
&self,
```

```
blob_id: &BlobId,
known_status: Option<BlobStatus>,
) -> ClientResult<(Epoch, Option<BlobStatus>)> {
let  committees = self.get_committees().await?;
let (epoch_to_be_read, blob_status) = if  committees.is_change_in_progress() {
// 에포크 변경 중인 경우 특별 처리
Self::continue_on_no_valid_status_received(
self.try_get_blob_status(blob_id, known_status).await,
&committees,
known_status,
).await?
} else {
// 정상 상태에서는 현재 에포크 사용
(
known_status
.map(|status| status.initial_certified_epoch())
.unwrap_or_else(|| Some(committees.epoch())),
known_status,
)
};


// 블롭이 등록되지 않은 경우 오류 반환
if  matches!(blob_status, Some(BlobStatus::Nonexistent | BlobStatus::Invalid { .
```

```
return Err(ClientError::from(ClientErrorKind::BlobIdDoesNotExist));
}

let current_epoch = committees.epoch();
let epoch_to_be_read = epoch_to_be_read.unwrap_or(current_epoch);

// 클라이언트가 뒤처진 경우 오류 반환
if epoch_to_be_read > current_epoch {
return Err(ClientError::from(ClientErrorKind::BehindCurrentEpoch {
client_epoch: current_epoch,
certified_epoch: epoch_to_be_read,
}));
}

Ok((epoch_to_be_read, blob_status))
}
```

2. 슬리버 수집과 RaptorQ 디코딩을 통한 원본 데이터 복원

2.1 슬리버 선택 메커니즘

```
pub(crate) struct SliverSelector<E: EncodingAxis> {
indices_and_shards: BiMap<SliverIndex, ShardIndex>,
```

```
_phantom: PhantomData<E>,
}


impl<E: EncodingAxis> SliverSelector<E> {
pub fn new(sliver_indices: &[SliverIndex], n_shards: NonZeroU16, blob_id: &Blo
let indices_and_shards = sliver_indices
.iter()
.map(|sliver_index| {
let pair_index = sliver_index.to_pair_index::<E>(n_shards);
let shard_index = pair_index.to_shard_index(n_shards, blob_id);
(*sliver_index, shard_index)
})
.collect::<BiMap<_, _>>();


Self {
indices_and_shards,
_phantom: PhantomData,
}
}
}
```

2.2 병렬 슬리버 검색

```
async fn retrieve_slivers<E: EncodingAxis>(
&self,
metadata: &VerifiedBlobMetadataWithId,
sliver_selector: &SliverSelector<E>,
certified_epoch: Epoch,
timeout_duration: Duration,
) -> ClientResult<Vec<SliverData<E>>>
where
SliverData<E>: TryFrom<Sliver>,
{
let blob_id = metadata.blob_id();
tracing::info!("starting to retrieve slivers {:?}", sliver_selector);
// 진행률 표시줄 생성
let progress_bar = styled_progress_bar(sliver_selector.len() as u64);
progress_bar.set_message(format!("requesting {} slivers", sliver_selector.len())


let committees = self.get_committees().await?;
let comms = self
.communication_factory
.node_read_communications(&committees, certified_epoch)?;


// 필요한 모든 슬리버에 대한 요청 생성
```

```
let futures = comms.iter().flat_map(|n| {
n.node
.shard_ids
.iter()
.cloned()
.filter(|&s| sliver_selector.should_read_from_shard(&s))
.map(|s| {
n.retrieve_verified_sliver::<E>(metadata, s)
.instrument(n.span.clone())
.inspect({
let value = progress_bar.clone();
move |result| {
if result.is_ok() {
value.inc(1);
}
}
})
})
});

let mut requests = WeightedFutures::new(futures);

// 적절한 동시성 제한으로 모든 요청 실행
```

```
requests
.execute_until(
&|_| false, // 모든 future를 실행
timeout_duration,
self.communication_limits
.max_concurrent_sliver_reads_for_blob_size(
metadata.metadata().unencoded_length(),
&self.encoding_config,
metadata.metadata().encoding_type(),
),
)
.await;


progress_bar.finish_with_message("slivers received");


let  slivers = requests
.take_results()
.into_iter()
.filter_map(|NodeResult { result, node, .. }| {
result
.map_err(|error| {
tracing::debug!(%node, %error, "retrieving sliver failed");
error
```

```
})
.ok()
})
.collect::<Vec<_>>();


Ok(slivers)
}
```

2.3 재시도 메커니즘

```
async fn retrieve_slivers_with_retry<E: EncodingAxis>(
&self,
metadata: &VerifiedBlobMetadataWithId,
sliver_indices: &[SliverIndex],
certified_epoch: Epoch,
max_attempts: usize,
timeout_duration: Duration,
) -> Result<Vec<SliverData<E>>, ClientError>
where
SliverData<E>: TryFrom<walrus_core::Sliver>,
{
let mut sliver_selector =
SliverSelector::<E>::new(sliver_indices, metadata.n_shards(), metadata.blob_id()
```

```
let mut attempts = 0;
let num_unique_slivers = sliver_selector.len();
let mut all_slivers = Vec::with_capacity(num_unique_slivers);
let start_time = Instant::now();
let mut last_error: Option<ClientError> = None;


while !sliver_selector.is_empty() {
// 타임아웃 또는 최대 재시도 횟수 확인
if start_time.elapsed() > timeout_duration {
tracing::debug!(
"timeout reached after {} while retrieving slivers",
humantime::Duration::from(timeout_duration)
);
break;
}
if attempts > max_attempts {
tracing::debug!("max attempts ({}) reached", max_attempts);
break;
}


tracing::debug!(?sliver_selector, "retrieving slivers");
match self
.retrieve_slivers(
```

```
metadata,
&sliver_selector,
certified_epoch,
timeout_duration - start_time.elapsed(),
)
.await
{
Ok(new_slivers) => {
// 성공적으로 검색된 인덱스 추적
for  sliver  in  new_slivers {
sliver_selector.remove_sliver(&sliver.index);
all_slivers.push(sliver);
}
}
Err(error) => {
tracing::warn!(?error, "error retrieving slivers");
last_error = Some(error);
}
}


attempts += 1;
if  all_slivers.len() != num_unique_slivers {
tokio::time::sleep(RETRIEVE_SLIVERS_RETRY_DELAY).await;
```

```
}
}

if all_slivers.len() != num_unique_slivers {
Err(ClientError::from(ClientErrorKind::Other(
format!(
"failed to retrieve some slivers ({}/{} successful): {:?}",
all_slivers.len(),
num_unique_slivers,
last_error
)
.into(),
)))
} else {
Ok(all_slivers)
}
}
```

2.4 RaptorQ 디코딩 과정

RaptorQ 디코딩은 walrus-core의 encoding 모듈에서 처리됩니다:

```
pub trait Decoder {
/// 슬리버들로부터 원본 데이터를 복원
```

```
fn decode(&self, slivers: &[Sliver]) -> Result<Vec<u8>, DecodingError>;
}

impl ReedSolomonDecoder {
pub fn decode_with_metadata(
&self,
slivers: &[SliverData<impl EncodingAxis>],
metadata: &BlobMetadata,
) -> Result<Vec<u8>, DecodingError> {
// 1. 충분한 슬리버가 있는지 확인
let required_symbols = self.config.source_symbols_for_axis();
if slivers.len() < required_symbols {
return Err(DecodingError::InsufficientSlivers);
}

// 2. 슬리버들을 심볼로 변환
let symbols: Vec<Symbol> = slivers
.iter()
.map(|sliver| sliver.to_symbols())
.flatten()
.collect();

// 3. RaptorQ 디코딩 수행
```

```
let decoded_data = self.raptorq_decode(&symbols)?;


// 4. 원본 크기로 자르기
let original_size = metadata.unencoded_length() as usize;
Ok(decoded_data[..original_size].to_vec())
}
}
```

3. 노드 장애 시 슬리버 복구 메커니즘

3.1 복구 심볼 시스템

```
pub struct RecoveryManager {
encoding_params: EncodingParams,
storage_client: StorageNodeClient,
}


impl RecoveryManager {
pub async fn recover_sliver(&self, sliver_id: SliverId) -> Result<Sliver, Rec
// 1. 필요한 복구 심볼 수 계산
let required_symbols = self.calculate_required_symbols(sliver_id.sliver_type);
// 2. 다른 노드들로부터 복구 심볼 수집
let recovery_symbols = self.collect_recovery_symbols(sliver_id, required_symbol
```

```
// 3. RaptorQ 디코딩으로 슬리버 복구
let recovered_sliver = self.decode_sliver(recovery_symbols)?;
Ok(recovered_sliver)
}

async fn collect_recovery_symbols(
&self,
sliver_id: SliverId,
required_count: usize,
) -> Result<Vec<RecoverySymbol>, RecoveryError> {
let mut recovery_symbols = Vec::new();
let mut contacted_nodes = 0;
let max_nodes = self.encoding_params.n_shards - self.encoding_params.f_byzantin

for node in self.available_nodes() {
if contacted_nodes >= max_nodes {
break;
}

match self.request_recovery_symbol(&node, &sliver_id).await {
Ok(symbol) => {
recovery_symbols.push(symbol);
if recovery_symbols.len() >= required_count {
```

```
break;
}
}
Err(e) => {
tracing::warn!("Failed to get recovery symbol from node {}: {}", node.id, e);
}
}
contacted_nodes += 1;
}


if  recovery_symbols.len() < required_count {
Err(RecoveryError::InsufficientRecoverySymbols {
required: required_count,
obtained: recovery_symbols.len(),
})
} else {
Ok(recovery_symbols)
}
}
}
```

3.2 Primary 슬리버 복구

Primary 슬리버 복구는 Secondary 슬리버들의 인코딩을 통해 이루어집니다:

```
async fn recover_primary_sliver(
&self,
blob_id: &BlobId,
target_pair_index: SliverPairIndex,
metadata: &VerifiedBlobMetadataWithId,
) -> Result<PrimarySliver, RecoveryError> {
// 1. Secondary 슬리버들로부터 복구 심볼 요청
let required_symbols = metadata.encoding_config().secondary_source_symbols();
let mut recovery_symbols = Vec::new();

for node in self.available_nodes() {
if recovery_symbols.len() >= required_symbols {
break;
}

// Secondary 슬리버를 Primary 인코딩으로 변환하여 복구 심볼 생성
match node.encode_secondary_for_primary_recovery(
blob_id,
target_pair_index,
).await {
Ok(symbol) => recovery_symbols.push(symbol),
```

```
Err(e) => tracing::warn!("Recovery symbol request failed: {}", e),
}
}

// 2. RaptorQ 디코딩으로 Primary 슬리버 복구
if recovery_symbols.len() >= required_symbols {
self.decode_primary_sliver(recovery_symbols, metadata)
} else {
Err(RecoveryError::InsufficientRecoverySymbols {
required: required_symbols,
obtained: recovery_symbols.len(),
})
}
}
```

3.3 Secondary 슬리버 복구

Secondary 슬리버 복구는 Primary 슬리버들로부터 직접 수행됩니다:

```
async fn recover_secondary_sliver(
&self,
blob_id: &BlobId,
target_pair_index: SliverPairIndex,
metadata: &VerifiedBlobMetadataWithId,
```

```
) -> Result<SecondarySliver, RecoveryError> {
// 1. Primary 슬리버들로부터 필요한 심볼들 수집
let required_symbols = metadata.encoding_config().primary_source_symbols();
let mut recovery_symbols = Vec::new();

for node in self.available_nodes() {
if recovery_symbols.len() >= required_symbols {
break;
}

// Primary 슬리버에서 해당 열의 심볼 추출
match node.get_primary_symbol_for_secondary_recovery(
blob_id,
target_pair_index,
).await {
Ok(symbol) => recovery_symbols.push(symbol),
Err(e) => tracing::warn!("Recovery symbol request failed: {}", e),
}
}

// 2. 충분한 심볼이 있으면 디코딩
if recovery_symbols.len() >= required_symbols {
self.decode_secondary_sliver(recovery_symbols, metadata)
```

```
} else {
Err(RecoveryError::InsufficientRecoverySymbols {
required: required_symbols,
obtained: recovery_symbols.len(),
})
}
}
```

4. 비잔틴 노드 탐지와 대응 전략

4.1 불일치 증명 시스템

```
pub  enum InconsistencyProof<A: EncodingAxis, P> {
/// 두 개의 서로 다른 슬리버가 같은 위치에 저장된 경우
ConflictingSlivers {
sliver1: SliverData<A>,
sliver2: SliverData<A>,
proof1: P,
proof2: P,
},
/// 슬리버가 메타데이터와 일치하지 않는 경우
InvalidSliver {
sliver: SliverData<A>,
```

```
proof: P,
expected_hash: Hash,
},
}


impl<A: EncodingAxis> InconsistencyProof<A, MerkleProof> {
pub fn verify(
&self,
metadata: &BlobMetadata,
encoding_config: &EncodingConfig,
) -> Result<(), InconsistencyError> {
match self {
Self::ConflictingSlivers { sliver1, sliver2, proof1, proof2 } => {
// 1. 두 슬리버가 같은 인덱스를 가지는지 확인
if sliver1.index != sliver2.index {
return Err(InconsistencyError::DifferentIndices);
}


// 2. 두 슬리버가 실제로 다른지 확인
if sliver1.data == sliver2.data {
return Err(InconsistencyError::IdenticalSlivers);
}
```

```
// 3. 각 슬리버의 머클 증명 검증
proof1.verify(&sliver1.hash(), &metadata.sliver_root(sliver1.index))?;
proof2.verify(&sliver2.hash(), &metadata.sliver_root(sliver2.index))?;

Ok(())
}
Self::InvalidSliver { sliver, proof, expected_hash } => {
// 1. 슬리버 해시가 예상과 다른지 확인
let  actual_hash = sliver.hash();
if  actual_hash == *expected_hash {
return Err(InconsistencyError::ValidSliver);
}

// 2. 머클 증명 검증
proof.verify(&actual_hash, expected_hash)?;

Ok(())
}
}
}
}
```

4.2 비잔틴 노드 탐지 알고리즘

```rust
pub struct ByzantineDetector {
encoding_config: EncodingConfig,
threshold: f32, // 의심스러운 행동의 임계값
}


impl ByzantineDetector {
pub async fn detect_byzantine_behavior(
&self,
blob_id: &BlobId,
responses: &[NodeResponse],
) -> Vec<SuspiciousNode> {
let mut suspicious_nodes = Vec::new();
// 1. 응답 시간 분석
let avg_response_time = self.calculate_average_response_time(responses);
let timeout_threshold = avg_response_time * 3.0;
// 2. 데이터 일관성 검사
let majority_data = self.find_majority_response(responses);
for response in responses {
let mut suspicion_score = 0.0;
// 응답 시간이 비정상적으로 긴 경우
if response.response_time > timeout_threshold {
suspicion_score += 0.3;
```

```
}
// 다수와 다른 데이터를 반환한 경우
if  let Some(ref  majority) = majority_data {
if  response.data != majority.data {
suspicion_score += 0.5;
}
}
// 잘못된 서명이나 증명을 제공한 경우
if !self.verify_response_authenticity(&response) {
suspicion_score += 0.8;
}
// 임계값을 초과하면 의심스러운 노드로 분류
if  suspicion_score >= self.threshold {
suspicious_nodes.push(SuspiciousNode {
node_id: response.node_id,
suspicion_score,
reasons: self.analyze_suspicion_reasons(&response, suspicion_score),
});
}
}
suspicious_nodes
}
```

```
fn find_majority_response(&self, responses: &[NodeResponse]) -> Option<&NodeRes
let mut response_counts = HashMap::new();
for response in responses {
let count = response_counts.entry(&response.data).or_insert(0);
*count += 1;
}
let majority_threshold = (responses.len() / 2) + 1;
response_counts
.iter()
.find(|(_, &count)| count >= majority_threshold)
.and_then(|(data, _)| {
responses.iter().find(|r| &r.data == data)
})
}
}
```

4.3 대응 전략

```
pub struct ByzantineResponseStrategy {
max_byzantine: usize,
recovery_manager: RecoveryManager,
}
```

```rust
impl ByzantineResponseStrategy {
pub  async  fn  handle_byzantine_nodes(
&self,
suspicious_nodes: Vec<SuspiciousNode>,
blob_id: &BlobId,
) -> Result<(), ByzantineHandlingError> {
// 1. 의심스러운 노드 수가 허용 한계를 초과하는지 확인
if  suspicious_nodes.len() > self.max_byzantine {
return Err(ByzantineHandlingError::TooManyByzantineNodes {
detected: suspicious_nodes.len(),
max_allowed: self.max_byzantine,
});
}

// 2. 각 의심스러운 노드에 대해 대응 조치 수행
for  suspicious_node  in  suspicious_nodes {
match  suspicious_node.suspicion_score {
score  if  score >= 0.8 => {
// 높은 의심도: 즉시 격리 및 불일치 증명 생성
self.quarantine_node(&suspicious_node.node_id).await?;
self.generate_inconsistency_proof(blob_id, &suspicious_node).await?;
}
score  if  score >= 0.5 => {
```

```
// 중간 의심도: 추가 검증 수행
self.perform_additional_verification(blob_id, &suspicious_node).await?;
}
_ => {
// 낮은 의심도: 모니터링만 수행
self.monitor_node(&suspicious_node.node_id).await?;
}
}
}

// 3. 손상된 데이터 복구
self.recover_corrupted_data(blob_id).await?;

Ok(())
}

async fn generate_inconsistency_proof(
&self,
blob_id: &BlobId,
suspicious_node: &SuspiciousNode,
) -> Result<(), ByzantineHandlingError> {
// 의심스러운 노드로부터 데이터 수집
let suspicious_data = self.collect_data_from_node(blob_id, &suspicious_node.nod
```

```
// 신뢰할 수 있는 노드들로부터 올바른 데이터 수집
let correct_data = self.collect_correct_data(blob_id).await?;
// 불일치 증명 생성
let inconsistency_proof = InconsistencyProof::ConflictingSlivers {
sliver1: suspicious_data,
sliver2: correct_data,
proof1: self.generate_merkle_proof(&suspicious_data).await?,
proof2: self.generate_merkle_proof(&correct_data).await?,
};
// 네트워크에 불일치 증명 제출
self.submit_inconsistency_proof(blob_id, &inconsistency_proof).await?;
Ok(())
}
}
```

5. 캐싱과 성능 최적화 기법

5.1 메타데이터 캐싱

```
pub struct MetadataCache {
cache: Arc<RwLock<LruCache<BlobId, CachedMetadata>>>,
ttl: Duration,
}
```

```rust
#[derive(Clone)]
struct CachedMetadata {
metadata: VerifiedBlobMetadataWithId,
cached_at: Instant,
access_count: AtomicU64,
}


impl MetadataCache {
pub  async  fn  get_or_fetch(
&self,
blob_id: &BlobId,
fetcher: impl Future<Output = Result<VerifiedBlobMetadataWithId, ClientError>>,
) -> Result<VerifiedBlobMetadataWithId, ClientError> {
// 1. 캐시에서 확인
{
let  cache = self.cache.read().await;
if  let Some(cached) = cache.get(blob_id) {
if  cached.cached_at.elapsed() < self.ttl {
cached.access_count.fetch_add(1, Ordering::Relaxed);
return Ok(cached.metadata.clone());
}
}
```

```
}

// 2. 캐시 미스 또는 만료된 경우 새로 가져오기
let metadata = fetcher.await?;
// 3. 캐시에 저장
{
let mut cache = self.cache.write().await;
cache.put(*blob_id, CachedMetadata {
metadata: metadata.clone(),
cached_at: Instant::now(),
access_count: AtomicU64::new(1),
});
}

Ok(metadata)
}
}
```

5.2 슬리버 캐싱 전략

```
pub struct SliverCache {
primary_cache: Arc<RwLock<LruCache<SliverKey, PrimarySliver>>>,
secondary_cache: Arc<RwLock<LruCache<SliverKey, SecondarySliver>>>,
```

```
cache_stats: CacheStats,
}

#[derive(Hash, Eq, PartialEq, Clone)]
struct SliverKey {
blob_id: BlobId,
pair_index: SliverPairIndex,
}

impl SliverCache {
pub  async  fn  get_primary_sliver(
&self,
key: &SliverKey,
) -> Option<PrimarySliver> {
let  cache = self.primary_cache.read().await;
if  let Some(sliver) = cache.get(key) {
self.cache_stats.record_hit();
Some(sliver.clone())
} else {
self.cache_stats.record_miss();
None
}
}
```

```rust
pub async fn put_primary_sliver(
&self,
key: SliverKey,
sliver: PrimarySliver,
) {
let mut cache = self.primary_cache.write().await;
cache.put(key, sliver);
}

// 캐시 효율성을 위한 지능적인 제거 정책
pub async fn evict_least_useful(&self) {
let mut primary_cache = self.primary_cache.write().await;
let mut secondary_cache = self.secondary_cache.write().await;

// 접근 빈도와 최근성을 고려한 제거
// LRU 캐시가 자동으로 처리하지만, 추가적인 메트릭 기반 제거도 가능
}
}
```

5.3 연결 풀링 최적화

```rust
pub struct ConnectionPool {
```

```
pools: HashMap<NodeId, Pool<HttpConnection>>,
max_connections_per_node: usize,
connection_timeout: Duration,
}

impl ConnectionPool {
pub  async  fn  get_connection(&self, node_id: NodeId) -> Result<PooledConnectio
let  pool = self.pools.get(&node_id).ok_or(NetworkError::NodeNotFound)?;
// 연결 풀에서 연결 가져오기 (타임아웃 적용)
tokio::time::timeout(self.connection_timeout, pool.get())
.await
.map_err(|_| NetworkError::ConnectionTimeout)?
.map_err(NetworkError::from)
}

pub  async  fn  create_pool_for_node(&mut  self, node_id: NodeId, endpoint: &str
let  pool = Pool::builder()
.max_size(self.max_connections_per_node as u32)
.connection_timeout(self.connection_timeout)
.build(HttpConnectionManager::new(endpoint))
.map_err(NetworkError::from)?;

self.pools.insert(node_id, pool);
```

```
Ok(())
}
}
```

5.4 배치 요청 최적화

```
pub struct BatchRequestOptimizer {
max_batch_size: usize,
batch_timeout: Duration,
}

impl BatchRequestOptimizer {
pub async fn batch_sliver_requests(
&self,
requests: Vec<SliverRequest>,
) -> Result<Vec<SliverResponse>, NetworkError> {
// 1. 노드별로 요청 그룹화
let mut requests_by_node: HashMap<NodeId, Vec<SliverRequest>> = HashMap::new()
for request in requests {
requests_by_node
.entry(request.node_id)
.or_default()
.push(request);
```

```
}

// 2. 각 노드에 대해 배치 요청 수행
let batch_futures: Vec<_> = requests_by_node
.into_iter()
.map(|(node_id, node_requests)| {
self.send_batch_to_node(node_id, node_requests)
})
.collect();

// 3. 모든 배치 요청을 병렬로 실행
let batch_results = futures::future::join_all(batch_futures).await;

// 4. 결과 수집 및 정리
let mut all_responses = Vec::new();
for result in batch_results {
match result {
Ok(responses) => all_responses.extend(responses),
Err(e) => tracing::warn!("Batch request failed: {}", e),
}
}

Ok(all_responses)
```

```
}

async fn send_batch_to_node(
&self,
node_id: NodeId,
requests: Vec<SliverRequest>,
) -> Result<Vec<SliverResponse>, NetworkError> {
// 배치 크기 제한 적용
let batches: Vec<_> = requests
.chunks(self.max_batch_size)
.map(|chunk| chunk.to_vec())
.collect();

let mut all_responses = Vec::new();
for batch in batches {
let batch_request = BatchSliverRequest {
node_id,
requests: batch,
timeout: self.batch_timeout,
};

match self.send_single_batch(batch_request).await {
Ok(responses) => all_responses.extend(responses),
```

```
Err(e) => {
tracing::warn!("Single batch failed for node {}: {}", node_id, e);
// 배치 실패 시 개별 요청으로 폴백
// (구현 생략)
}
}
}

Ok(all_responses)
}
}
```

5.5 압축 최적화

```
pub struct CompressionOptimizer {
compression_threshold: usize, // 압축을 적용할 최소 크기
compression_level: u32,
}

impl CompressionOptimizer {
pub fn compress_sliver_if_beneficial(
&self,
sliver: &SliverData<impl EncodingAxis>,
```

```
) -> Result<CompressedSliver, CompressionError> {
let  original_size = sliver.serialized_size();
// 작은 슬리버는 압축하지 않음
if  original_size < self.compression_threshold {
return Ok(CompressedSliver::Uncompressed(sliver.clone()));
}

// zstd 압축 적용
let  compressed_data = zstd::encode_all(
sliver.as_bytes(),
self.compression_level as i32,
)?;

// 압축 효과가 있는 경우만 압축된 버전 사용
if  compressed_data.len() < original_size * 9 / 10 { // 10% 이상 절약
Ok(CompressedSliver::Compressed {
data: compressed_data,
original_size,
compression_ratio: original_size  as f32 / compressed_data.len() as f32,
})
} else {
Ok(CompressedSliver::Uncompressed(sliver.clone()))
}
```

```
}

pub fn decompress_sliver(
&self,
compressed: &CompressedSliver,
) -> Result<Vec<u8>, CompressionError> {
match compressed {
CompressedSliver::Uncompressed(sliver) => Ok(sliver.as_bytes().to_vec()),
CompressedSliver::Compressed { data, original_size, .. } => {
let decompressed = zstd::decode_all(data.as_slice())?;
// 크기 검증
if decompressed.len() != *original_size {
return Err(CompressionError::SizeMismatch {
expected: *original_size,
actual: decompressed.len(),
});
}
Ok(decompressed)
}
}
}
}
```

6. 성능 메트릭 및 모니터링

6.1 검색 성능 메트릭

```rust
pub struct RetrievalMetrics {
// 지연시간 메트릭
pub blob_retrieval_duration: HistogramVec,
pub sliver_retrieval_duration: HistogramVec,
pub metadata_retrieval_duration: HistogramVec,
// 처리량 메트릭
pub blobs_retrieved_total: CounterVec,
pub slivers_retrieved_total: CounterVec,
pub bytes_retrieved_total: CounterVec,
// 오류 메트릭
pub retrieval_errors_total: CounterVec,
pub timeout_errors_total: CounterVec,
pub byzantine_detections_total: CounterVec,
// 캐시 메트릭
pub cache_hits_total: CounterVec,
pub cache_misses_total: CounterVec,
pub cache_evictions_total: CounterVec,
}
```

```
impl RetrievalMetrics {
pub fn record_blob_retrieval(&self, duration: Duration, success: bool, blob_si
let result = if success { "success" } else { "failure" };
self.blob_retrieval_duration
.with_label_values(&[result])
.observe(duration.as_secs_f64());
if success {
self.blobs_retrieved_total
.with_label_values(&[])
.inc();
self.bytes_retrieved_total
.with_label_values(&[])
.inc_by(blob_size);
} else {
self.retrieval_errors_total
.with_label_values(&["blob_retrieval"])
.inc();
}
}
}
```

결론

Walrus의 데이터 검색 및 복구 시스템은 RedStuff 인코딩 알고리즘을 기반으로 한 강력하고 효율적인 메커니즘을 제공합니다. 블롭 ID를 통한 체계적인 검색 과정, RaptorQ 디코딩을 통한 안정적인 데이터 복원, 비잔틴 장애 허용을 위한 복구 메커니즘, 그리고 다양한 성능 최적화 기법들이 통합되어 높은 가용성과 내구성을 보장합니다.

특히 슬리버 기반의 분산 저장과 복구 시스템은 개별 노드 장애나 비잔틴 공격에도 불구하고 데이터의 무결성과 접근성을 유지할 수 있도록 설계되었습니다. 캐싱, 연결 풀링, 배치 처리 등의 최적화 기법들은 실제 운영 환경에서의 성능과 효율성을 크게 향상시킵니다.